



TrasmUM

Sistema de gestão integrada para uma rede de lojas de conveniência

Oradores

José Lourenço Fernandes

Simão Oliveira

Estrutura da Apresentação

- **Introdução** — Tema e contexto do projeto
- **Processo de Desenvolvimento** — As 4 etapas de desenvolvimento
- **Engenharia de requisitos** — Etapas de levantamento e refinamento de requisitos
- **Arquitetura da Solução** — Decisões técnicas e ADRs
- **Implementação** — Processo de implementação
- **Verificação e Validação** — Testes unitários e manuais
- **Reflexão sobre utilização de IA** — Vantagens, limitações e aprendizagens
- **Conclusões** — O que levámos deste projeto

Introdução

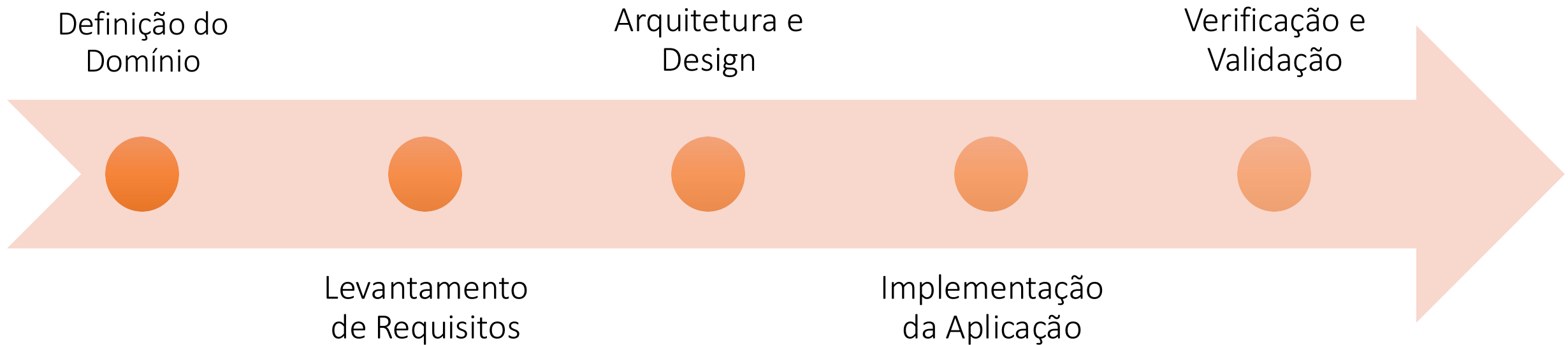
TEMA

Sistema de gestão integrada para uma cadeia de pequenas lojas de conveniência

Objetivos

- Registo e processamento de dados de vendas, produtos, stocks e fornecedores
- Consolidação de informação num sistema central de dados
- Gestão e controlo de inventário, reposição de produtos nas lojas, faturação e geração de relatórios de gestão

Processo de Desenvolvimento





Do caos analógico a uma rede coordenada

Três lojas de conveniência em Trás-os-Montes, operadas ineficientemente — adquiridas pela administração e em modernização.

PROBLEMAS DO SISTEMA ATUAL

- 1 Rastreamento de inventário em cadernos e folhas soltas — ruturas e desperdício de perecíveis
- 2 Consolidação de informações (vendas, pagamentos, etc...) por telefone com cada loja — moroso e propenso a erros
- 3 Faturas e guias de remessa guardadas em caixas de sapatos — extravio e atrasos
- 4 Lojas isoladas, sem partilha de informação entre estabelecimentos

OBJETIVOS DA SOLUÇÃO

- 1 Registo digital de vendas com atualização automática de stock
- 2 Fecho de dia automatizado com consolidação no servidor central
- 3 Arquivo digital de documentos e rastreabilidade dos pagamentos
- 4 Autonomia plena de cada loja + supervisão estratégica centralizada

Levantamento e especificação de requisitos

- Simulação de entrevistas com stakeholder
- Geração user stories
- Levantamento e refinamento de requisitos
- Criação de documentação formal de especificação de requisitos
- Geração de casos de uso e diagramas de máquina de estados

Registo de Entrevista - Elicitação de Requisitos

Entrevistado(a):	Hélder Cerqueira (Administração)
Equipa/Entrevistador:	Equipa de Análise de Sistemas
Data e Hora:	02 de Março de 2026, 16:00
Local/Plataforma:	Instalações da Sede - TrasmUM
Objetivo Principal:	Compreender as ineficiências do modelo atual de gestão de fornecedores e apuramento de requisitos.

Questionário de Levantamento de Requisitos — Operadores de Caixa

Ola, Simão. Quando submeter este formulário, o proprietário verá o seu nome e endereço de e-mail.

1. Que dificuldades encontra atualmente no registo manual de vendas? [?]

Introduza a sua resposta

2. Com que frequência ocorrem erros no apuramento de valores de caixa? [?]

☐ Nunca

☐ Raramente

☐ Ocasionalmente

☐ Frequentemente

☐ Muito frequentemente

3. Considera útil que o sistema atualize automaticamente o stock após cada venda? [?]

☐ Nada útil

☐ Pouco útil

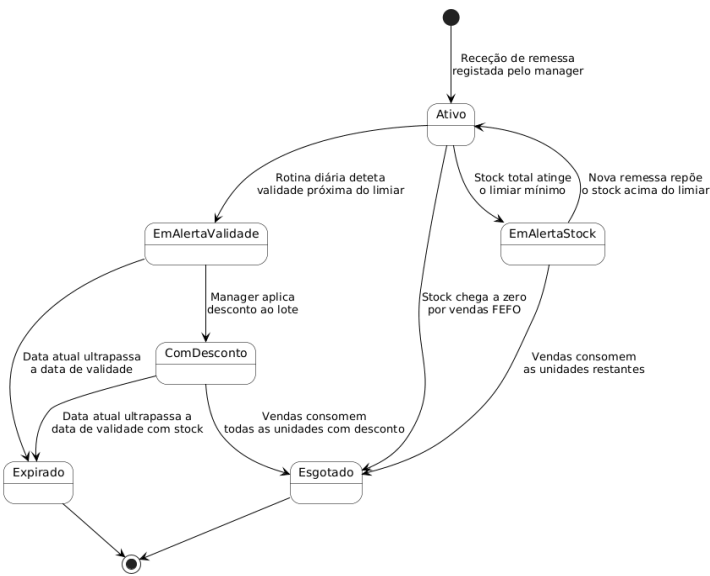
☐ Útil

☐ Muito útil

☐ Essencial

4. Tem alguma sugestão adicional para melhorar o processo de vendas e fecho de caixa? [?]

Introduza a sua resposta



Transcrição da Entrevista

P: Boa tarde, Hélder. Para começar, gostaríamos de saber quais são as suas principais expectativas para o novo sistema que iremos desenvolver.

R: Boa tarde! Olhem, acima de tudo quero que o sistema nos traga controlo e confiança nos números, porque hoje em dia isso simplesmente não existe. Quero conseguir ver as vendas e o stock das lojas sem depender de telefonemas nem de papelada perdida. Também é importante que ajude a equipa nas lojas a trabalhar mais rápido e com menos erros — os operadores perdem demasiado tempo com coisas que deviam ser automáticas. No fundo, procuro uma solução simples de usar, mas robusta o suficiente para suportar o crescimento da rede.

P: No que diz respeito ao funcionário, como é que este deve, num dia normal de funcionamento, interagir com o sistema?

R: Bom, o funcionário chega, autentica-se no sistema — e aqui é importante que seja apenas um por loja, por turno, para sabermos sempre quem é responsável pelo quê — e durante o dia basicamente regista as vendas e trata de eventuais devoluções. No final do dia, o manager faz o fecho de dia e o sistema encarrega-se de enviar tudo automaticamente para a sede. Sem telefonemas, sem papelada — é isso que queremos.

Requisitos e Casos de Uso

RESULTADO DA ESPECIFICAÇÃO

23

Requisitos Funcionais

18

Requisitos Não Funcionais

14

Casos de Uso

3

Atores hierárquicos

Requisito de Utilizador	O manager deve registar a receção de remessas introduzindo a respetiva guia, de forma a atualizar o stock local com lotes independentes e gerar um registo de pagamento pendente associado ao fornecedor.
Analista	José Fernandes, Gabriel Dantas, Luís Ferreira, Simão Oliveira
Fonte	Entrevista — Hélder Cerqueira
Área do Sistema	Terminal de Loja
Requisitos de Sistema	1. O sistema deve permitir ao manager a introdução dos dados da guia de remessa (fornecedor, produtos, quantidades, data, valor) e a sua submissão local. 2. O sistema deve criar registos independentes de stock (novos lotes) para as unidades recebidas, atribuindo-lhes um código único de lote e a data de validade da entrega, sem sobrepor as validades dos lotes pré-existentes. 3. O sistema deve gerar localmente um registo de pagamento pendente associado ao fornecedor, com base no valor financeiro registado na guia, para inclusão no pacote de fecho de dia. 4. O sistema deve permitir ao manager atualizar o estado de um registo de pagamento pendente para “pago”, ficando essa alteração incluída no próximo pacote de fecho de dia.
Relevância para a Aplicação	Elimina o arquivo físico caótico em caixas de sapatos, assegura a rastreabilidade individual dos lotes e garante que as obrigações de tesouraria ficam registadas localmente para posterior consolidação no Servidor Central.

Caso de uso	Consultar Dashboards e Relatórios Analíticos
Requisitos de Origem	RF20, RF21
Ator	CEO
Área do Sistema	Terminal de Servidor Central
Descrição	Consulta de painéis de controlo com métricas globais de vendas e stocks, e de relatórios analíticos avançados que identificam produtos em risco de rutura iminente ou com excesso de stock parado, com base nos dados consolidados recebidos via fecho de dia.
Pré-condição	O CEO tem sessão iniciada com perfil CEO e existem dados consolidados de pelo menos um fecho de dia recebido.
Pós Condição	O CEO visualiza as métricas e relatórios.
Fluxo normal	1. A Cátia acede ao módulo de análise e seleciona a visualização pretendida (<i>dashboards</i> ou relatórios analíticos). 2. Consoante a seleção, o sistema compila estatísticas macro para apresentação (<i>dashboard</i>), aplica algoritmos para gerar listas de produtos em risco (relatório analítico), ou gera um relatório estruturado exportável com os dados filtrados. Em qualquer das vistas, o sistema disponibiliza filtros para refinar os dados por loja, período temporal e categoria de produto.
Fluxo de exceção	[Ausência de dados consolidados suficientes] (passo 2) 2.1. O sistema apresenta mensagem informativa indicando a ausência de dados históricos suficientes para a análise solicitada.

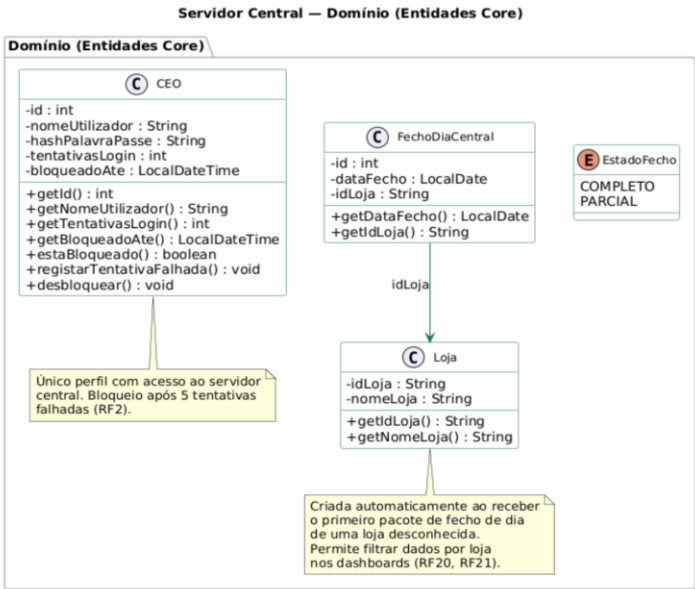
Arquitetura e Design do Software

Interação com o LLM dividiu-se em 4 fases:

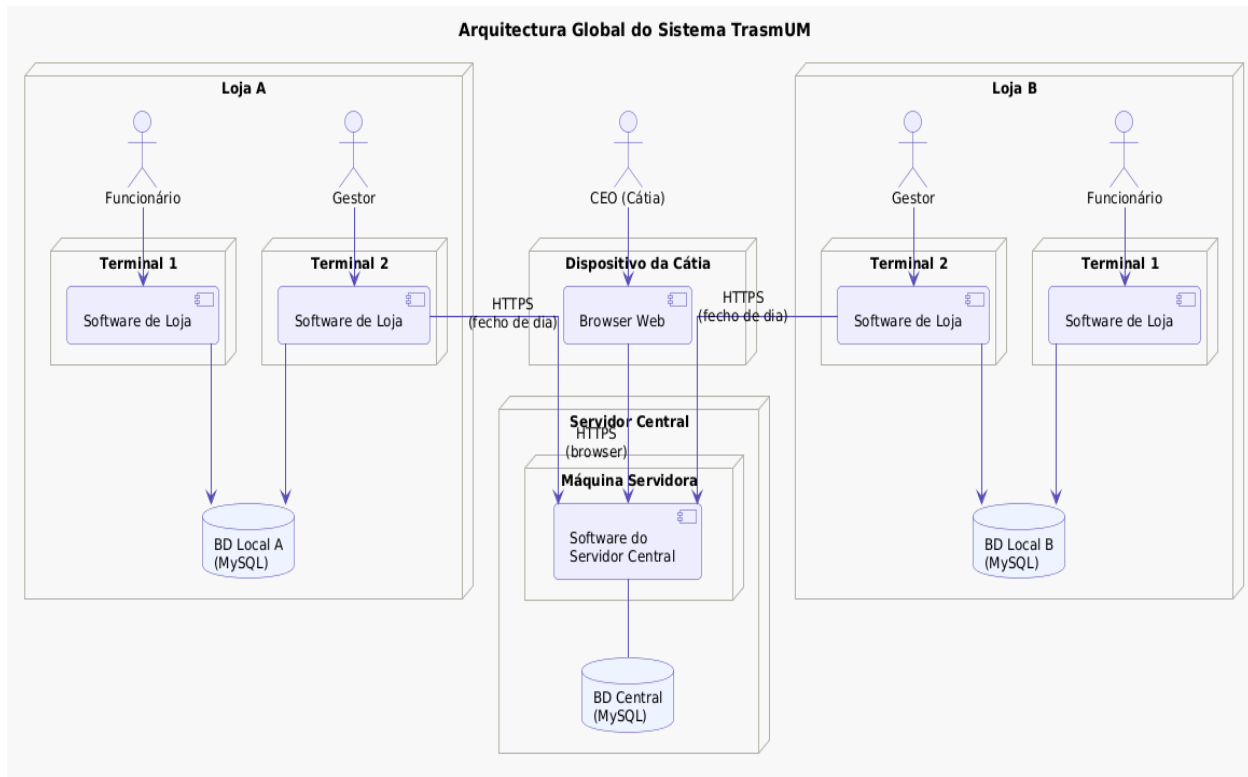
- **Definição arquitetural** — Solicitação de sugestões arquiteturais adequados ao sistema. Especificação detalhada da sugestão elegida pela equipa
- **Revisão independente** — O modelo atuou como arquiteto revisor independente, identificando vulnerabilidades e cenários de falha não cobertos
- **Diagramas técnicos** — Geração assistida de diagramas de componentes, classes , sequência e modelos de BD em PlantUML
- **Esboço de interfaces** — agente via MCP gerou automaticamente wireframes no Pencil a partir das Design Guidelines

Ecrã: ECR-03 — Painel Principal (Manager)

Campo	Detalhe
Perfil(is)	Manager
Objetivo	Hub de navegacao alargado para Manager: todas as funcionalidades de Funcionario mais gestao de inventario, fornecedores e fecho de dia.
Dados Exibidos	Nome do utilizador; loja; hora; alertas de stock minimo pendentes; alertas de validade proxima; resumo do dia (numero de vendas, valor acumulado).
Ações Disponíveis	Navegar para todas as areas (vendas, devolucoes, remessas, catalogo, fornecedores, pagamentos, utilizadores, configuracoes, fecho de dia); Terminar Sessao.
Componentes shadcn/ui	Sidebar (com separadores por categoria); Card (KPI resumo); Badge (alertas); NavigationMenu; Button.
Ecrã(s) Anterior(es)	ECR-01 (apos login)
Ecrã(s) Posterior(es)	ECR-06 (Caixa) ECR-11 (Devolucao) ECR-13 (Produtos) ECR-14 (Pedidos Remessa) ECR-15 (Rececao Remessa) ECR-17 (Catalogo) ECR-19 (Documentos Fornecedores) ECR-20 (Pagamentos) ECR-22 (Desconto Validade) ECR-24 (Utilizadores) ECR-26 (Configuracoes) ECR-27 (Fecho de Dia Loja)



Arquitetura federada — porquê dois softwares



CLIENTE-SERVIDOR ASSIMÉTRICO

Cada loja é uma unidade computacional autónoma. O servidor central recebe os dados, mas nunca emite instruções para as lojas.

Software de Loja

JavaFX · MySQL local · offline-first

Terminal de ponto de venda. Suporta vendas, devoluções, gestão de stock, remessas, catálogo, utilizadores e fecho de dia.

Software do Servidor Central

Javalin · Vue.js · MySQL agregada

Aplicação web. Recebe pacotes de fecho de dia e disponibiliza dashboards à CEO.

Decisões arquiteturais documentadas

Todas as decisões de design foram registadas formalmente em ADRs (Architecture Decision Records), preservando contexto, alternativas consideradas, justificação e consequências de cada escolha.

DESTAQUES

Decisões com maior impacto técnico no sistema final

ADR-01

Arquitetura federada

Cliente–servidor assimétrico, com cada loja como unidade autónoma

ADR-02

Dois softwares independentes

Loja e Servidor com ciclos de vida desacoplados

ADR-04

Delta Sync com 3 estados

PENDENTE → EM_TRÂNSITO → CONFIRMADO, com reversão automática

ADR-05

Idempotência no servidor

INSERT ... ON DUPLICATE KEY UPDATE absorve reenvios sem duplicar

ADR-08

HTTPS com certificado próprio

Cifra do canal de fecho de dia, sem custos de CA externa

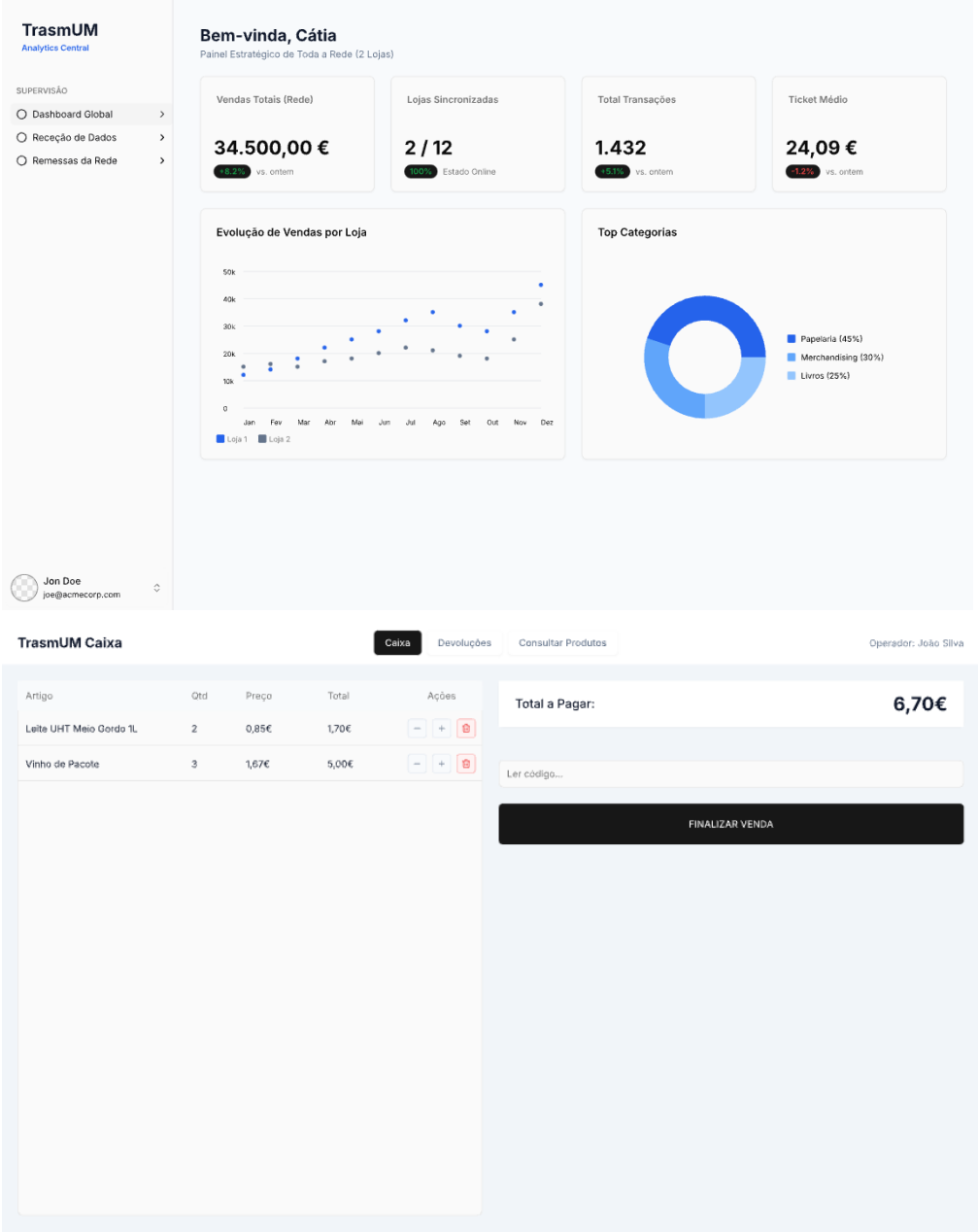
ADR-09

Layered Architecture + MVC

Quatro camadas com dependências estritamente unidirecionais

Esboço das Interfaces do Sistema

- Fornecimento dos requisitos a uma LLM que produziu uma lista de design guidelines
- Conexão do agente (Gemini) à aplicação Pencil por meio de um endpoint MCP para esboço da interface gráfica
- Permitiu utilizar prompts para fazer alterações diretamente no esboço criado



Implementação

- Arquitetura, ADRs e artefactos de design entregues ao Claude Sonnet como base
- Claude Sonnet gerou os prompts de implementação entregues ao Claude Code
- Criação de instruções estruturadas para a implementação pelo Claude Code
- Revisão de código e teste manual do comportamento da aplicação

Descrição	Prompt fornecido ao Claude Code na continuação da sessão do Prompt 1, após confirmação de compilação. Cobre a camada de serviço completa com lógica de negócio detalhada, o gateway de sincronização HTTP, a raiz de composição completo, todos os controllers JavaFX, os ficheiros FXML e a estilização CSS. A lógica crítica de negócio — FEFO, quatro guardas da devolução, Delta Sync — é especificada passo a passo para evitar que o modelo infira comportamentos incorretos.
LLM Utilizado	Claude Code (Anthropic)
Prompt	Contexto: continuação directa do Prompt 1. O projeto já tem estrutura Maven compilável, schema SQL, classes de domínio, interfaces e implementações de repositório, DatabaseConnection e AppContext esqueleto. Interfaces de serviço: IAutenticacaoServico (autenticar, encerrarSessao, verificarSessaoDuplicada), IAutorizacaoServico (exigirPerfil, temPerfil), IAuditoriaServico (registrar, obterLogsPendentes), IVendaServico (iniciarVenda, adicionarLinha, finalizarVenda, anularVenda, calcularTroco, obterVendasPendentes), IDevolucaoServico (processar, obterDevolucoesPendentes), ICaixaServico (abrirSessao, registrarSangria, fecharSessao, verificarLimite, obterSessoesPendentes), ICatalogoServico (criarProduto, editarProduto, desativarProduto, aplicarDesconto, gerarAlertasStockMinimo, gerarAlertasValidade, pesquisarProduto), IRemessaServico (registrarRemessa, associarPedidoPendente, criarPedidoRemessa, listarPedidos, obterRemessasPendentes, obterPagamentosPendentes), IPagamentoServico (listarPendentes, liquidar), IUtilizadorServico (criar, desativar, editar, validarGestorAtivo), IFechoDiaServico (executarFecho, reenviar, reverterEmTransito). Lógica crítica dos serviços: VendaServico - iniciarVenda cria Venda em memória sem persistir; adicionarLinha busca produto por código de barras, busca lotes FEFO, decrementa stock no lote, cria LinhaVenda; finalizarVenda persiste venda+linhas+fatura em transação, atualiza saldo da SessaoCaixa, verifica limite de caixa, regista log VENDA; anularVenda repõe stock nos lotes e persiste com estado ANULADA. DevolucaoServico - quatro guardas em sequência: (1) busca fatura, se null lança FaturaNaoEncontradaException; (2) verifica se artigo consta na fatura; (3) verifica unidades disponíveis para devolução; (4) busca lote por dataValidadeEmbalagem, se válido repõe stock. FechoDiaServico - recolhe pendentes de todos os serviços, marca EM_TRANSITO, compila PacoteFechoDTO, envia via SincronizacaoGateway; em falha reverte para PENDENTE; em sucesso marca CONFIRMADO exclusivamente após ACK. Estrutura de ecrãs JavaFX: LoginView.fxml, MainView.fxml com menu lateral adaptativo ao perfil, VendaView.fxml, DevolucaoView.fxml, CatalogoView.fxml, CaixaView.fxml, RemessaView.fxml, PagamentoView.fxml, UtilizadorView.fxml, FechoDiaView.fxml. Navegação por substituição do centro do BorderPane principal. Tarefa: implementar todas as interfaces e implementações de serviço, PacoteFechoDTO e DTOs de sincronização, HttpSincronizacaoGateway, AppContext completo, todos os controllers JavaFX, todos os ficheiros FXML, app.css, e MainApp como ponto de entrada. Confirmar que o projeto compila, arranca e que o fluxo login → venda → fecho de dia funciona end-to-end.
Fontes Fornecidas	Prompt gerado na Entrada 46, código produzido pelo Prompt 1 (Entrada 47), mockups de interface da fase de design.
Reflexão Final	O código gerado foi revisto pela equipa. Dado que o JavaFX não suporta exportação a partir do Pencil, o frontend da loja ficou ligeiramente diferente dos esboços definidos na fase de design. Os desvios identificados foram documentados no DESIGN_CHANGES.md.

Verificação e Validação

- Geração de testes unitários
- Verificação dos requisitos funcionais e não funcionais através de testes unitários e validação manual
- Revisão humana crítica

Req.	Descrição Resumida	Verificação	Estado
RNF1	Persistência de venda em ≤ 2 s.	M	Passou
RNF2	Geração de fatura em ≤ 3 s.	M	Passou
RNF3	Envio do pacote de fecho em ≤ 5 min.	M	Passou
RNF4	<i>Dashboards</i> carregam em ≤ 3 s.	M	Passou
RNF5	<i>Backups</i> automáticos diários com retenção de 30 dias.	—	Não implementado
RNF6	Autenticação individual obrigatória para todos os perfis.	UT + M	Passou
RNF7	Bloqueio após 5 tentativas falhadas consecutivas; desbloqueio automático configurável.	UT + M	Passou
RNF8	Palavras-passe armazenadas com BCrypt (hash irreversível).	UT + M	Passou
RNF9	Acesso a dados financeiros consolidados restrito ao perfil CEO.	UT + M	Passou
RNF10	Registo de logs de auditoria para todas as ações críticas.	UT + M	Passou
RNF11	Idempotência da transmissão — reenvio não duplica registos na base de dados central.	M	Passou
RNF12	Fluxo de registo de venda em ≤ 5 interações.	M	Passou
RNF13	Mensagens de erro descritivas com indicação de ação corretiva.	M	Passou
RNF14	Suporte a crescimento de 50% de volume sem degradação superior a 10%.	—	Não testado
RNF15	Registo de novas lojas sem reconfiguração manual do servidor central.	UT + M	Passou

Reflexão sobre utilização de IA - Vantagens

- **Aceleração do desenvolvimento** — rascunhos competentes em minutos; fundação de ambos os softwares gerada em poucas iterações
- **Qualidade da documentação** — registo académico consistente, terminologia uniforme e formatação coerente entre secções
- **Foco na engenharia** — esforço da equipa deslocado da produção mecânica para a revisão crítica e decisão de engenharia
- **Deteção de inconsistências** — revisão por LLM identificou contradições internas e cenários de falha não cobertos



Reflexão sobre utilização de IA - Desvantagens

- **Custo de contexto** — prompts verdadeiramente úteis exigem artefactos, decisões e restrições prévias — investimento não negligenciável
- **Deriva de coerência** — em sessões longas, o modelo introduz inconsistências que requerem reconciliação progressiva
- **Complacência automatizada** — outputs plausíveis e bem estruturados reduzem a atenção crítica na revisão
- **Tendência para sobre-engenharia** — abstrações injustificadas, camadas intermédias e padrões desnecessários surgiram em todas as fases



Conclusões

- **LLMs são úteis quando a tarefa é bem definida** — documentação extensa, exploração arquitetural, revisão de inconsistências, geração de testes com contrato claro
- **LLMs são contraproducentes quando** — se exige simplicidade radical, o domínio requer conhecimento tácito, ou o prompt custa mais do que a tarefa
- **A abordagem correta é criteriosa** — fase a fase, tarefa a tarefa, sempre com revisão humana atenta





TrasmUM

Sistema de gestão integrada para uma rede de lojas de conveniência

Oradores

José Lourenço Fernandes

Simão Oliveira